

Bhagiratha: An AI-Enabled Geospatial System for Village Pond Site Selection and Catchment Delineation

RAHUL SEERA, Indian Institute of Technology Bhilai, India

Village ponds (*Amrit Sarovars*) are frequently excavated at locations chosen for administrative convenience rather than hydrology, and consequently either stay dry or breach during the monsoon. This paper presents **Bhagiratha**, a web-based geospatial decision-support system that automates pond site selection, upstream catchment delineation, runoff estimation and pond dimensioning. The system ingests either raster Digital Elevation Models (SRTM 30 m via OpenTopography) or vector contour maps (KML/KMZ), interpolates contours onto a regular grid with SciPy's Delaunay-based linear interpolation, conditions the surface with Priority-Flood depression filling, and routes flow with the D8 algorithm on a latitude-corrected metric grid. The pond site is chosen as the interior cell of maximum flow accumulation. Ten years of daily precipitation from the Open-Meteo archive yield the June–September monsoon depth, from which seasonal runoff volume is estimated with the volumetric form of the Rational Method ($Q = C \cdot I \cdot A$, $C = 0.35$); the pond is sized to hold 70% of that volume at a depth between 2.5 m and 4.5 m. The application is an asynchronous FastAPI modular monolith with a Leaflet.js front end, PostgreSQL/PostGIS persistence with an automatic file-based fallback, a rounded-grid response cache, and a watchdog supervisor for a shared campus container. On a real 1 m contour survey from Chhattisgarh (1,355 contours), the system selects a site at 21.24171°N, 81.28692°E draining a 3.9116 km² catchment, estimates 1.71×10^6 m³ of monsoon runoff, and completes end-to-end in about 3 s.

CCS Concepts: • **Applied computing** → **Earth and atmospheric sciences**; • **Information systems** → *Geographic information systems*.

Additional Key Words and Phrases: Hydrological Modeling, Catchment Delineation, D8 Flow Direction, Rational Method, Village Pond Planning, GIS, FastAPI, Leaflet.js

1 Introduction and Problem Statement

Under national water-conservation programmes such as *Mission Amrit Sarovar*, village administrators must choose where to excavate community ponds. Manual site selection suffers from three recurring problems:

- (1) **Subjective siting**: ponds are placed on available land rather than on natural drainage lines, so little runoff reaches them.
- (2) **Unknown catchment**: the upstream area that drains to a candidate site cannot be reliably delineated by hand over several square kilometres.
- (3) **Hydraulic mismatch**: without linking catchment area to historical rainfall, ponds are either too small (overflow and embankment breach) or too large (never fill).

Bhagiratha lets a non-expert user click a point, draw a candidate land parcel, or upload a contour map, and returns (1) a recommended pond location, (2) the delineated catchment boundary, (3) the expected monsoon runoff volume, (4) recommended depth, surface area and capacity, and (5) a 0–100 suitability score, all overlaid on satellite imagery.

2 System Architecture

Four quality attributes drive the architecture: *correctness* of the hydrological result, *interactive latency* (seconds, not minutes, per analysis), *resilience* on a shared container with one CPU, no database service and partial internet access, and *deployability* as a single process with no mandatory infrastructure. Following the HLD, Bhagiratha is an asynchronous **modular monolith**: one FastAPI process whose modules are separated by narrow, typed interfaces (Pydantic models

Author's Contact Information: Rahul Seera, Indian Institute of Technology Bhilai, Durg, Chhattisgarh, India, rahulseera.r2@gmail.com.

Manuscript submitted to ACM

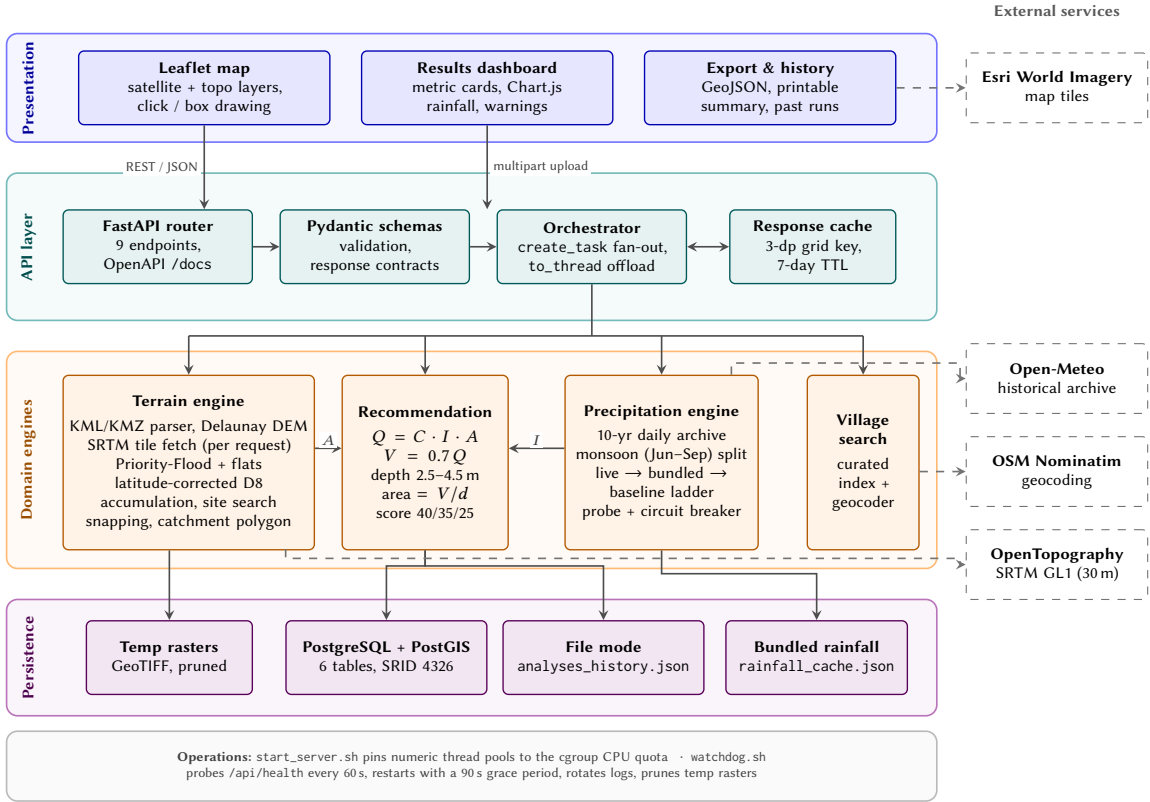


Fig. 1. Layered component architecture. *A*: catchment area and slope; *I*: monsoon rainfall depth. Solid arrows are in-process calls; dashed arrows are network calls to external services, each backed by a local fallback (§2.3).

in, Pydantic models out), so each engine can be tested, replaced or moved to its own service without touching the others.

Figure 1 shows the resulting layers.

- The **presentation layer** is a single-page Leaflet application. It holds no domain logic: it renders whatever the API returns, including the warnings array.
- The **API layer** validates every request against a schema, consults the response cache, and orchestrates the engines. I/O-bound calls run as concurrent `asyncio` tasks, and CPU-bound work is moved to worker threads.
- Four **domain engines** carry the logic. The terrain engine turns a DEM or contour map into a pond site and catchment; the precipitation engine turns a location into a 10-year monsoon rainfall depth; the recommendation engine combines them into runoff, dimensions and a score; village search resolves place names.
- The **persistence layer** is chosen at startup: PostGIS when reachable, otherwise JSON files. Bundled reference data and temporary rasters live beside it.
- An **operations** layer of two shell scripts supervises the process.

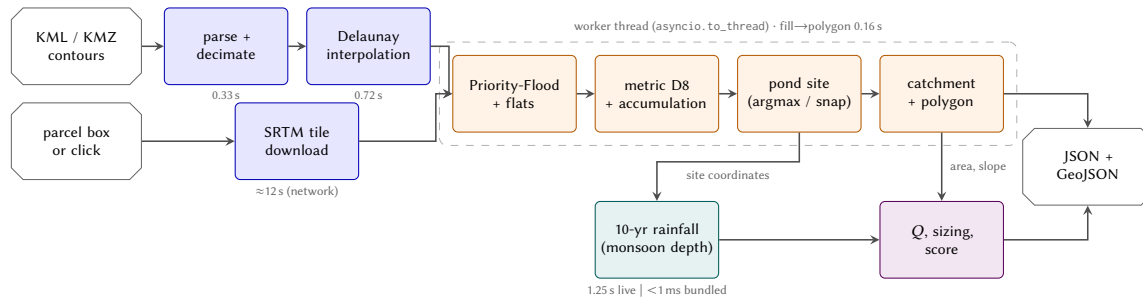


Fig. 2. Analysis pipeline with measured stage latencies (reference survey, development machine). Flow routing runs in a worker thread; where Open-Meteo is unreachable, rainfall comes from the bundled archive.

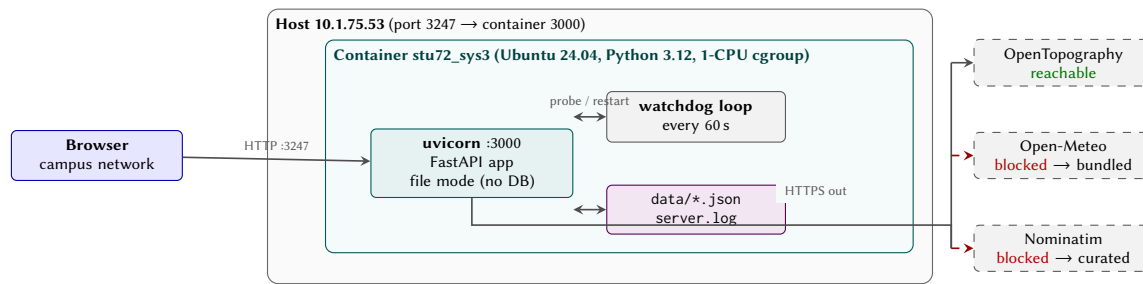


Fig. 3. Deployment view of the evaluation container, with the reachability of external services observed on 27 September 2026.

2.1 Request Lifecycle

Figure 2 traces a contour-map request, the most complex path. The multipart upload is validated by type and size, then parsed and interpolated to a grid in a worker thread (about 1 s together). The terrain engine conditions the grid, routes flow on a metric D8 grid, selects the pond cell and delineates its catchment in 0.16 s. Only then is the rainfall query issued, because it needs the pond coordinates. Its result, with the catchment area and slope, feeds the recommendation engine, and the response returns both the numbers and the GeoJSON polygon.

Point and land-area requests replace parsing and interpolation with an SRTM download (about 12 s, the dominant cost). In point mode the site is known up front, so the DEM download and rainfall query are started together and the rainfall latency disappears behind the download. A repeated point request is answered from the rounded-grid cache in milliseconds.

2.2 Deployment View

Figure 3 shows the production deployment. The container exposes port 3000 through host port 3247. uvicorn runs the application in file mode, and a shell loop runs the watchdog every 60 s. Three properties of this environment, all discovered by measurement rather than assumed, shaped the design (§9.1): the cgroup grants one CPU while the OS reports 120 cores; Open-Meteo and Nominatim are blocked while OpenTopography is reachable; and the disk is 99% full.

2.3 Failure Handling and Graceful Degradation

Every external dependency has a local fallback, and every fallback is announced to the user in the response’s warnings array. The system never fails silently and never returns an HTTP 500 because a dependency is missing.

PostgreSQL unavailable → persistent-file mode, selected by a 3 s connection test at startup; history lives in `data/analyses_history.json`.

Open-Meteo unavailable → bundled 10-year archive statistics for the nearest point within 11 km, then a conservative regional baseline. A background probe and a circuit breaker keep this decision off the request path, so no user request waits for a blocked service.

OpenTopography unavailable → one retry on connection failure. Point mode then returns HTTP 502 with a clear message; land-area mode switches to synthetic demonstration terrain, explicitly labelled as not real.

Nominatim unavailable → the curated village index only.

Catchment truncated by the DEM edge → a larger tile is fetched once, and the area is reported as a lower bound if the catchment is still cut off.

Process crash or hang → the watchdog’s health probe fails and the server is restarted. A 90 s grace period prevents it from killing a server that is still starting.

2.4 Design Decisions and Trade-offs

- **Bundled reference data vs. live queries only:** live Open-Meteo data is preferred, but a small, reproducible snapshot (`scripts/build_rainfall_cache.py`) makes results on a blocked network identical to results on an open one, instead of silently different.
- **Thread pools sized to the real CPU budget:** numeric libraries size their thread pools from the visible core count, not the cgroup quota; pinning them to one thread reduced contour analysis on the container from 28–49 s to under 1 s.
- **Monolith vs. microservices (FastAPI [8]):** flow routing manipulates in-memory raster matrices (152×200 for the reference contour grid, up to $\approx 865 \times 865$ for the largest SRTM tile). Serialising these between services would add latency and memory copies for no benefit at this scale, so the terrain, rainfall and recommendation modules run in-process behind narrow function interfaces.
- **Asynchronous concurrency:** in point mode the terrain and rainfall pipelines depend only on (lat, lon) , so they are started concurrently with `asyncio.create_task`. CPU-bound work (contour parsing, interpolation, flow routing) runs in worker threads via `asyncio.to_thread` so the event loop, and the watchdog’s `/api/health` probe, stay responsive during long analyses.
- **Spatial relational storage:** catchments are polygons, so PostgreSQL with PostGIS provides native geometry types (`GEOMETRY(. . . , 4326)`) through GeoAlchemy2.
- **Rounded-grid caching:** point analyses are cached under a key of latitude and longitude rounded to 3 decimals (≈ 110 m), with a 7-day TTL in the database or a bounded (500-entry) in-memory dictionary when the database is absent. A cache hit returns in ≈ 8 ms versus ≈ 13 s cold.
- **Database never blocks startup:** `is_db_available()` tests the connection with a 3 s timeout. If PostgreSQL is unreachable (as on the evaluation container), the service runs in *persistent-file mode*, storing history in `data/analyses_history.json`.

3 Hydrological Methodology and Core Algorithms

3.1 Contour Parsing and Grid Interpolation

For KML/KMZ uploads, `km1_parser.py`:

- (1) **Unpacks KMZ** archives in memory and reads the first `.kml` entry.
- (2) **Extracts coordinates and elevation** from every `<Placemark>` containing a `<LineString>`. Elevation is read from `<name>` (as a number, or via the regex `[+]?[0-9]+\.[0-9]+` for labels such as “Contour 280m”), falling back to the mean of the coordinates’ `z` values.
- (3) **Decimates** dense inputs with a deterministic stride $\lfloor N/N_{max} \rfloor$ ($N_{max} = 20,000$), which bounds the interpolation input to below $2N_{max}$ points; the reference file’s 159,113 vertices become 22,731.
- (4) **Interpolates** the scattered (x_i, y_i, z_i) onto a regular grid (200 cells along the longer axis) with SciPy’s `griddata` in linear mode (barycentric interpolation on a Delaunay triangulation). Cells outside the convex hull are filled by nearest-neighbour interpolation so the grid has no gaps.
- (5) **Writes a GeoTIFF** (EPSG:4326) whose pixel *centres* coincide exactly with the interpolation sample points (origin shifted by half a pixel, spacing $\text{span}/(n - 1)$).

3.2 Flow Routing and Catchment Delineation

The same pipeline (`catchment.py`, built on PySheds [6]) runs on SRTM tiles and on contour-derived grids:

- (1) **Type normalisation:** elevations are converted to `float32` with no-data cells set to `NaN`, so integer SRTM tiles and floating-point contour grids follow one code path.
- (2) **Depression filling:** PySheds’ `fill_depressions`, an implementation of the Priority-Flood algorithm [1], raises every closed depression to its spill elevation so that every cell drains to the grid boundary.
- (3) **Flat resolution:** `resolve_flats` adds small gradients across perfectly flat areas so flow direction is defined everywhere.
- (4) **Latitude-corrected D8 routing [2]:** each cell drains to the neighbour with the steepest descent

$$S_k = \frac{z_{i,j} - z_k}{d_k}, \quad k \in \{1, \dots, 8\}, \quad (1)$$

where d_k is the metric distance between cell centres. Because the raster is in geographic coordinates, a degree of longitude is shorter than a degree of latitude, so the flow grid is given metric cell sizes

$$\Delta x = \Delta \lambda \cdot 111,320 \cdot \cos \phi, \quad \Delta y = \Delta \phi \cdot 111,320, \quad (2)$$

evaluated at the grid’s central latitude ϕ . At 21°N , routing on raw degree spacing would under-weight east–west slopes by $\approx 7\%$; on the reference survey the correction changes 759 of 30,400 flow directions (2.5%).

- (5) **Flow accumulation:** $A_{flow}(i, j)$ is the number of cells (including itself) whose D8 path passes through (i, j) .
- (6) **Automatic pond-site discovery** (contour and land-area modes): the pond is placed at the interior cell of maximum accumulation,

$$(i^*, j^*) = \arg \max_{(i,j) \in \Omega} A_{flow}(i, j), \quad (3)$$

where Ω excludes a border margin of $\delta = 8\%$ of the rows and columns on each side, since accumulation near a DEM edge is distorted by flow leaving the grid. In land-area mode Ω is further restricted to cells inside the user’s parcel, while flow is routed over a surrounding buffer so the upstream catchment is not cut off at the

parcel boundary. The reported coordinate is the centre of cell (i^*, j^*) . “Best” here is purely hydrological (the point of maximum flow convergence); the criteria the method does *not* consider are listed in §9.2.

- (7) **Pour-point snapping** (point mode): a clicked location is rarely exactly on the one-cell-wide D8 channel, and a click one cell off it traces only a hillslope. The pour point is therefore moved to the highest-accumulation cell within 2 cells (≈ 60 m on SRTM), the standard “snap pour point” step.
- (8) **Delineation and vectorisation**: the D8 tree is traced upstream from the pour cell (by index, avoiding coordinate round-off) to obtain a boolean catchment mask, which is polygonized into GeoJSON. On the reference survey the result is a single valid, simple polygon.
- (9) **Area, slope and truncation check**: area is the cell count times the metric cell area; mean slope is the gradient magnitude (in metric units) averaged over the catchment. If the catchment touches the raster border, it may extend beyond the DEM: point mode then re-fetches a larger tile ($\pm 0.12^\circ$ instead of $\pm 0.05^\circ$), and every mode warns that area and runoff are lower bounds if the catchment still reaches the edge.

3.3 Runoff Estimation

Seasonal runoff volume is estimated with the volumetric form of the Rational Method [3, 4]:

$$Q = C \cdot I \cdot A, \quad (4)$$

- A : catchment area in m^2 ($\text{km}^2 \times 10^6$);
- I : mean June–September (monsoon) rainfall *depth* in metres (mm/1000), averaged over the last 10 complete calendar years of Open-Meteo daily data [7];
- $C = 0.35$: runoff coefficient for rural agricultural land on loamy soil, a typical textbook value within the ≈ 0.2 – 0.5 range tabulated for cultivated land.

Because I is a seasonal depth rather than a design intensity, Q is a seasonal volume (m^3), not the peak discharge (m^3/s) the classical Rational Method computes.

3.4 Pond Dimensioning

The pond is sized to retain 70% of seasonal runoff, $V_{\text{target}} = 0.70 Q$, with the remaining 30% discharged through a spillway. The depth d is scaled progressively with V_{target} and clamped to [2.5, 4.5] m: 2.5 m is the minimum that limits proportional evaporation loss under tropical summers, and 4.5 m is the practical limit for unlined earthen embankments:

$$d = \begin{cases} 2.5 & V_{\text{target}} \leq 5,000 \text{ m}^3 \\ 2.5 + 2.0 \cdot \frac{V_{\text{target}} - 5,000}{195,000} & 5,000 < V_{\text{target}} < 200,000 \text{ m}^3 \\ 4.5 & V_{\text{target}} \geq 200,000 \text{ m}^3 \end{cases} \quad A_{\text{surface}} = \frac{V_{\text{target}}}{d}. \quad (5)$$

3.5 Suitability Score

Per HLD §3.5, the score combines three sub-scores (each 0–100) with weights 40/35/25 and is clamped to [5, 98]:

$$S = \text{clip}_{[5,98]} (0.40 S_{\text{catch}} + 0.35 S_{\text{slope}} + 0.25 S_{\text{rain}}). \quad (6)$$

All three sub-scores are continuous and piecewise linear:

- **Catchment adequacy** S_{catch} (area a in km^2): $10 \rightarrow 50$ for $a < 0.1$; $50 \rightarrow 100$ for $0.1 \leq a < 0.5$; 100 in the optimal band $0.5 \leq a \leq 5.0$; then -3 per km^2 with a floor of 50 for larger catchments, where spillway and breach risk grow.
- **Slope suitability** S_{slope} (mean slope s in %): 100 for $s \leq 4$; $\max(10, 100 - 12(s - 4))$ above, reaching 88 at 5% and penalising steeper ground where excavation and embankment stability suffer.
- **Rainfall reliability** S_{rain} (annual mean P in mm): 100 for $P \geq 1000$; $60 \rightarrow 100$ for $500 \leq P < 1000$; $\max(15, 60P/500)$ below 500 mm.

4 System Implementation and Component Mapping

Table 1 maps each functional requirement to its implementation, and Table 2 maps the design to Computer System Design course themes.

5 API Specification

All nine routes are served by `app/main.py`; interactive OpenAPI documentation is generated at `/docs`. Request bodies are validated by Pydantic (`app/schemas.py`); invalid bodies return HTTP 422 with FastAPI's standard error array. Table 3 summarises the routes.

5.1 Request and Response Payloads

Point analysis (POST `/api/analyze`):

```
Request: {"lat": 21.2500, "lon": 81.2900, "village_id": null}
Response: {"request_id": 505628834, "lat": 21.25, "lon": 81.29,
  "terrain": {"catchment_polygon": {"type": "Polygon",
    "coordinates": [...]}},
  "area_km2": 20.0068, "avg_slope": 3.05},
  "rainfall": {"annual_avg_mm": 1415.2,
    "seasonal": {"monsoon_mm": 1251.4, "non_monsoon_mm": 163.8},
    "data_years": 10},
  "recommendation": {"runoff_m3": ..., "depth_m": 4.5,
    "surface_area_m2": ..., "capacity_m3": ...,
    "suitability_score": ...},
  "warnings": []}
```

Land-area analysis (POST `/api/analyze-area`):

```
Request: {"bounds": {"min_lat": 21.235, "max_lat": 21.255,
  "min_lon": 81.278, "max_lon": 81.298},
  "name": "Selected Land Area"}
Response: {"selected_bounds": {...},
  "suggested_pond_site": {"lat": ..., "lon": ...},
  "catchment_area_km2": ..., "avg_slope_percent": ...,
  "catchment_polygon": {GeoJSON},
  "expected_water_volume_m3": ...,
  "rainfall": {...}, "recommendation": {...},
  "warnings": [...]}
```

Contour analysis (POST `/analyzeContour`, uploaded as multipart field `contour_map`): the response for the reference file is reproduced in §8.

Warnings. Degraded results are never silent. The warnings array reports (a) use of bundled archive rainfall or the regional baseline when Open-Meteo is unreachable, (b) use of synthetic demonstration terrain in land-area mode when OpenTopography is unreachable, (c) catchments that reach the DEM or contour-map edge (area is a lower bound), and (d) flow-analysis failure in land-area mode (rough whole-parcel estimate). Point-mode responses carrying warnings are not cached.

6 Data Model and Persistence

6.1 PostGIS Schema

`app/db/models.py` defines six tables (SRID 4326 for all geometries); `Base.metadata.create_all` creates them at startup when the database is reachable.

Each point analysis writes one `analysis_request` row and one row in each of the three result tables (one-to-one via `request_id`). Land-area and contour analyses are recorded in the file-based history described below.

6.2 Resilient File-Based Fallback

The evaluation container has no PostgreSQL service. The request-scoped database dependency yields `None` when the database is unavailable, and every endpoint then uses:

- `data/analyses_history.json`: a JSON array (newest first, at most 100 records) of `{request_id, village_id, created_at, lat, lon, area_km2, avg_slope, runoff_m3, depth_m, surface_area_m2, capacity_m3, suitability_score}`, loaded at startup and rewritten after each analysis;
- an in-memory rounded-grid response cache (at most 500 entries, oldest evicted first).

`GET /api/analyze/{id}` served from this file returns the stored figures without the polygon or rainfall detail (which are not persisted) and says so in warnings.

7 External Data Services

- **OpenTopography Global DEM API** (`portal.opentopography.org/API/globaldem`): `demtype=SRTMGL1` (1 arc-second, ≈ 30 m) [5], bounding box south/north/west/east, `outputFormat=GTiff`, API key from `.env`. Tiles are $\pm 0.05^\circ$ around a point (retried at $\pm 0.12^\circ$ if the catchment is truncated) or the parcel plus a 0.01° buffer. Timeout 20 s. Each request writes to its own temporary file, deleted after use.
- **Open-Meteo Historical Weather API** (`archive-api.open-meteo.com/v1/archive`): `daily=precipitation_sum` from 1 January ten years ago to 31 December of last year, `timezone=auto`; no key needed. Timeout 15 s. On failure (or while the circuit breaker is open) the app uses *bundled archive data*: `data/rainfall_cache.json`, built by `scripts/build_rainfall_cache.py` from the same API, holds the 10-year statistics for 19 points around Durg-Bhilai (the demo sites plus a 0.1° grid), and the nearest point within 11 km is used. Beyond that it uses the regional baseline (1150 mm/yr, 920 mm monsoon, reported with `data_years = 0`). Either way a note is returned.
- **OpenStreetMap Nominatim** (`nominatim.openstreetmap.org/search`): used only when the 37-entry curated list of Chhattisgarh and Andhra Pradesh villages yields fewer than 8 matches; query suffixed with “, India”, 3 s timeout, identifying User-Agent; failures are ignored.

8 Experimental Evaluation

8.1 Reference Contour Survey

The system was evaluated on contours_1m.kml, a real 1 m contour survey of a rural watershed near Durg-Bhilai, Chhattisgarh. The results below were obtained on 27 September 2026 through the /analyzeContour endpoint with live Open-Meteo data (2016–2025):

- **Input:** 1,355 contour lines, 159,113 vertices, elevations 267–298 m (31 m relief), extent $0.0312^\circ \times 0.0238^\circ$ ($\approx 3.2 \times 2.6$ km).
- **Grid:** 152×200 cells of $16.3 \text{ m} \times 17.5 \text{ m}$.
- **Pond site:** 21.24171°N , 81.28692°E , the interior cell of maximum accumulation.
- **Catchment:** 3.9116 km^2 (13,719 cells), mean slope 3.99%; a single valid, non-self-intersecting polygon whose vector area (3.9114 km^2) matches the cell-count area. The catchment does not touch the grid border, so it is not truncated.
- **Rainfall:** mean annual 1415.2 mm, of which 1251.4 mm (88%) falls in June–September.
- **Runoff:** $Q = 0.35 \times 1.2514 \text{ m} \times 3.9116 \times 10^6 \text{ m}^2 = 1,713,242 \text{ m}^3$.
- **Pond:** target capacity $0.70 Q = 1,199,269 \text{ m}^3$, depth 4.5 m, surface area $266,504 \text{ m}^2$ (≈ 26.7 ha).
- **Suitability:** 98.0/100 (all three sub-scores at 100; clamped to 98).

The evaluation container cannot reach Open-Meteo (§9.1); it uses the bundled copy of the same archive data (§7) and reproduces these figures exactly, with a note naming the data source. Where no bundled point is within 11 km, the regional baseline (920 mm monsoon) is used instead; at this site that would give $Q = 1,259,535 \text{ m}^3$.

8.2 Scientific Verification Audit

Before submission, every stage was checked against the requirements in the project guidelines, which led to the following corrections. Table 5 compares the reference results before and after.

- (1) **Latitude-corrected D8.** PySheds takes cell spacing from the raster’s affine transform, which is in degrees for EPSG:4326, so the correction $\Delta x = \Delta \lambda \cdot 111,320 \cos \phi$ had only been applied to area and slope, not to flow direction. D8 now runs on a grid with metric cell sizes.
- (2) **Georeferencing.** Contour-derived GeoTIFFs placed pixel corners, not centres, at the interpolation sample points, and the reported pond coordinate was a cell corner. Pixel centres now coincide with the samples and the reported site is a cell centre, shifting the site by one cell (≈ 16 m) and increasing the area by 1.0%.
- (3) **Real SRTM tiles.** PySheds’ compiled Priority-Flood failed on the `int16` elevations of SRTM tiles, so land-area mode silently fell back to a rough “45% of the parcel” estimate and point mode returned HTTP 502. Elevations are now normalised to `float32`.
- (4) **Parcel-constrained siting.** Land-area mode searched the whole downloaded tile, which extends beyond the parcel, so the pond could be placed outside the user’s land. The search is now restricted to the parcel.
- (5) **Pour-point snapping and truncation detection** were added (§3).
- (6) **Scoring.** The catchment sub-score was not flat over the $0.5\text{--}5.0 \text{ km}^2$ optimal band and jumped from 50 to 60.8 at 0.1 km^2 ; it is now continuous and flat over the band.

- (7) **No silent fallbacks.** The contour endpoint used a different, undisclosed rainfall baseline (940 mm) without a warning; one baseline is now shared by all endpoints and always reported. The land-area synthetic-terrain warning now states plainly that the terrain is not real.
- (8) **Error handling.** The database dependency caught exceptions raised by the endpoint and yielded again, turning HTTP 502 responses into unhandled 500 errors when PostgreSQL was connected.
- (9) **Deployment.** The evaluation container was running an older revision, with a placeholder score and a binary 2.5/4.5 m depth; the verified code is now deployed. The container-specific fixes are described in §9.1.

Nine regression tests (`tests/test_scientific.py`) cover unit conversions, depth bounds, score continuity, `int16` DEMs, parcel containment, pixel-centre alignment, the rainfall circuit breaker, bundled-rainfall lookup, and a blocked-network run of the reference survey that must reproduce the live-rainfall result. The full suite (27 tests) passes.

8.3 Generalisation and Robustness Tests

The automated suite exercises: compressed KMZ archives (identical results to the KML); malformed XML, empty files and wrong extensions (clean HTTP 400); contours labelled with free text or only 3-D coordinates; land-area selections; the cache and database layers; and synthetic `int16` SRTM-like tiles with no-data cells. Land-area analysis of the parcel $[21.235, 21.255]^{\circ}\text{N} \times [81.278, 81.298]^{\circ}\text{E}$ on real SRTM data returns a site inside the parcel with a 7.09 km^2 catchment.

8.4 Performance

Median of three runs on the development machine (Python 3.12, campus network): Download time from OpenTopography dominates point and land-area requests; the hydrological computation itself takes well under a second.

9 Deployment, Limitations and Future Work

9.1 Evaluation Container

The service runs at <http://10.1.75.53:3247/> in container `stu72_sys3` (Ubuntu 24.04, Python 3.12.3), with container port 3000 mapped to host port 3247. `uvicorn` is started by `scripts/start_server.sh` and supervised by a loop that runs `scripts/watchdog.sh` every 60 s. The watchdog probes `/api/health` (6 s timeout), restarts the server on failure, prunes temporary rasters older than 30 min and rotates `server.log` beyond 20 MB. Observed conditions on 27 September 2026:

- **Disk:** the shared overlay filesystem was 99% full (3.8 GB free of 247 GB); the project occupies 848 MB.
- **Network:** OpenTopography is reachable from the container, but Open-Meteo and Nominatim are not. Analyses on the container therefore use bundled archive rainfall within the demo region (the regional baseline elsewhere), always with a note, and village search uses only the curated list.
- **Database:** no PostgreSQL service; the server runs in persistent-file mode.
- **CPU:** the cgroup grants one CPU (`cpu.max = 100000 100000`), but `nproc` reports the host's 120 cores. NumPy, OpenBLAS and Numba therefore each started ≈ 120 threads that contended for a single CPU, and flow routing took 26–42 s instead of 0.15 s. `start_server.sh` now pins `OMP/OPENBLAS/MKL/NUMBA_NUM_THREADS` to 1, cutting `/analyzeContour` on the container from 28–49 s to 0.7–0.9 s, including the first request after a restart.

Two further hardening measures came from observing the container. Because Open-Meteo is blocked there and each attempt stalled for up to 45 s, rainfall queries now have a 12 s deadline and a 10-minute circuit breaker, fed by a background probe that checks Open-Meteo at startup and every 5 minutes, so no user request waits on the blocked

service. A background warm-up on a tiny synthetic DEM compiles the terrain kernels at startup. And because the server takes about 20 s to import its libraries on this CPU, the watchdog now skips restarts for a process younger than 90 s, so it no longer kills a server that is still starting up.

9.2 What the System Decides, and What It Does Not

The three modes answer different questions:

- **Point analysis** *evaluates* a location chosen by the user (snapped to the strongest drainage line within ≈ 60 m): “if a pond is built here, how much water reaches it, how large should it be, and how suitable is it?” It does not search for a better site.
- **Land-area selection** *chooses* the cell of maximum flow accumulation inside the user’s parcel.
- **Contour-map analysis** *chooses* the cell of maximum flow accumulation over the whole map, excluding the 8% border.

In the choosing modes, “best” means only *where the most surface runoff converges*. The recommendation is a hydrological screening result, not a final siting decision. The system has no information about, and therefore does not check:

- **Existing water bodies.** Rivers, canals, tanks and reservoirs are the strongest convergence lines, so the maximum-accumulation cell can fall inside one. On the reference survey the selected site lies in the channel of the Shivnath river: hydrologically the correct convergence point, but a location for a check dam or barrage rather than an excavated pond. OpenStreetMap maps the river, its banks and four canals in this area, so excluding such cells from the candidate set is a direct extension (§9.4).
- **Land ownership and availability:** whether the land is government, panchayat or private, and whether it is free of encroachment. Land-area mode lets an administrator restrict the search to land known to be available, but the parcel boundary is the user’s responsibility.
- **Land use and structures:** fields, homesteads, roads, bridges, schools, cremation grounds or sacred sites at or near the site.
- **Soil and geology:** permeability (sandy soils lose water to seepage), rock depth, and suitability of excavated soil for embankments.
- **Groundwater and water quality:** water-table depth, recharge potential, and pollution sources upstream.
- **Flood and dam safety:** backwater from a nearby river, spillway design for extreme events, and risk to people downstream.
- **Downstream users and approvals:** water rights of downstream villages, irrigation-department permissions, and environmental and forest clearances.
- **Access, cost and community need:** approach roads for machinery, excavation cost, and where the water is actually needed.

Every recommendation must therefore be verified by a field visit and by the revenue, irrigation and engineering authorities before excavation. The system’s role is to narrow the search to hydrologically sound candidates and to size them consistently.

9.3 Limitations

- **Pond scale.** Holding 70% of the runoff from a 3.9 km² catchment requires a 26.7 ha pond, far larger than a typical village pond. For large catchments the recommendation is better read as the harvestable potential, to be split across several ponds or check dams.
- **Constant runoff coefficient.** $C = 0.35$ ignores land cover, soil type and antecedent moisture; a land-use/soil lookup would improve Q .
- **DEM resolution.** SRTM 30 m cannot resolve the fine channels a 1 m contour survey captures. Near the reference site, point mode on SRTM snaps to a larger stream than the contour survey's catchment, so SRTM-based results should be treated as screening estimates.
- **Single outlet.** Only the maximum-accumulation cell is proposed; ranking several candidate sites (supported by the suitability score) is future work.
- **Water balance.** Evaporation, seepage and demand are not modelled; the 2.5–4.5 m depth bounds are engineering rules of thumb.
- **Synthetic fallback.** If OpenTopography is unreachable, land-area mode uses synthetic terrain; results are clearly marked as unsuitable for siting decisions.

9.4 Future Work

- **Water-body exclusion:** remove OpenStreetMap rivers, canals and ponds (buffered) from the candidate set, and warn when a user-selected point falls in water.
- **Multi-site ranking:** report the top- k candidates separated by a minimum distance, each with its own score, and split large catchments across several smaller ponds or check dams.
- **Land-use and soil layers:** a runoff coefficient from land cover and soil type, and exclusion of built-up areas.
- **Cadastral integration:** restrict candidates to government or panchayat land parcels where such records are available.
- **Water balance:** evaporation and seepage losses to check how long the pond holds water after the monsoon.

10 AI Usage Disclosure

In accordance with course academic-integrity guidelines, the author discloses that LLM coding assistants (Google DeepMind Antigravity and Anthropic Claude) were used during development: for handling KML namespace edge cases, structuring the SciPy interpolation routine, debugging Numba compilation issues in PySheds, and for a pre-submission audit that identified the corrections listed in §8. All algorithms, formulas, design decisions and tests were reviewed and validated by the author.

11 Deliverables

- **Source code:** <https://github.com/RahulSeera/pond-planning-backend>
- **Live application:** <http://10.1.75.53:3247/> (IIT Bhilai campus network)
- **Demonstration video:** *[YouTube link to be added after upload]*

References

- [1] R. Barnes, C. Lehman, and D. Mulla. 2014. Priority-flood: An optimal depression-filling and watershed-labeling algorithm for digital elevation models. *Computers & Geosciences* 62 (2014), 117–127.

- [2] J. F. O'Callaghan and D. M. Mark. 1984. The extraction of drainage networks from digital elevation data. *Computer Vision, Graphics, and Image Processing* 28, 3 (1984), 323–344.
- [3] E. Kuichling. 1889. The relation between the rainfall and the discharge of sewers in populous districts. *Transactions of the American Society of Civil Engineers* 20 (1889), 1–56.
- [4] V. T. Chow, D. R. Maidment, and L. W. Mays. 1988. *Applied Hydrology*. McGraw-Hill, New York.
- [5] T. G. Farr et al. 2007. The Shuttle Radar Topography Mission. *Reviews of Geophysics* 45, 2 (2007), RG2004.
- [6] M. Bartos. 2020. pysheds: Simple and fast watershed delineation in Python. <https://github.com/mbartos/pysheds>.
- [7] Open-Meteo. Historical Weather API. <https://open-meteo.com/en/docs/historical-weather-api>.
- [8] S. Ramirez. FastAPI. <https://fastapi.tiangolo.com>.

Table 1. Functional component mapping.

Functional Requirement	Source File(s)	Key Classes / Methods	Responsibilities
Satellite imagery & interactive map	app/static/index.html app/static/js/app.js app/static/css/style.css	initMap(), renderResults(), renderMapOverlays(), renderRainfallChart(), exportGeoJSON()	Esri World Imagery and topographic layers, area drawing tool, metric cards, Chart.js rainfall chart, GeoJSON export.
API routing & orchestration	app/main.py app/schemas.py	analyze(), analyze_area_endpoint(), analyze_contour(), health_check()	Pydantic validation, concurrent task dispatch, worker-thread offloading, HTTP error mapping, static file serving.
Contour map ingestion	app/modules/kml_parser.py	parse_contours(), load_kml_bytes()	KMZ unzipping, LineString extraction, elevation from name or z values.
Contour-to-DEM interpolation	app/modules/dem_from_contours.py	contours_to_grid(), write_grid_to_geotiff()	Point decimation, Delaunay linear interpolation, nearest-neighbour hull fill, centre-aligned GeoTIFF.
Suitable-site identification & catchment delineation	app/modules/catchment.py	find_pond_site(), delineate_catchment(), catchment_to_geojson()	Priority-Flood fill, flat resolution, metric D8, accumulation, interior/parcel pour point, snapping, polygonization, truncation flag.
Land-area selection	app/modules/area_analysis.py	analyze_land_area(), generate_terrain_for_bounds()	SRTM tile around parcel, parcel-restricted site search, synthetic-terrain fallback with warning.
Point terrain & DEM fetch	app/modules/terrain.py app/modules/dem_fetch.py	get_catchment(), fetch_dem_tile()	OpenTopography download to per-request temp file, truncation retry.
Historical rainfall	app/modules/rainfall.py	get_rainfall_stats(), aggregate_rainfall(), fallback_rainfall()	10-year Open-Meteo query, monsoon/non-monsoon aggregation, climate baseline.
Runoff, sizing & scoring	app/modules/recommendation.py	recommend()	$Q = C \cdot I \cdot A$, 70% capture, depth scaling, surface area, suitability score.
Village search	app/modules/village_search.py	search_villages()	Curated village index plus Nominatim fallback.
Persistence & caching	app/db/database.py, models.py, crud.py, cache.py	is_db_available(), AnalysisRequest, save_terrain_result(), get_cached_response()	PostGIS schema, GeoAlchemy2 bindings, history queries, 7-day rounded-grid cache; JSON-file fallback in main.py.
Process supervision	scripts/start_server.sh scripts/watchdog.sh	shell daemon	Health probe every 60 s, restart on failure, log rotation at 20 MB, temp-raster pruning after 30 min.

Table 2. Mapping to Computer System Design (CSD) themes.

CSD Theme	Course Topic	Application in Bhagiratha
Concurrency & latency	Asynchronous I/O, event loops, thread offloading	<code>asyncio.create_task</code> runs the DEM download (≈ 12 s) and rainfall query (≈ 1.2 s) concurrently, hiding the rainfall latency entirely. <code>asyncio.to_thread</code> moves CPU-bound parsing, interpolation and flow routing off the event loop: during a 13.5 s land-area analysis, <code>/api/health</code> still answered in ≈ 2 ms.
Scalability & caching	Spatial key caching, amortisation	Responses keyed on coordinates rounded to 3 decimals (≈ 110 m cells) with a 7-day TTL; a repeat point query drops from ≈ 13 s to ≈ 8 ms. The in-memory cache (500 entries) and history (100 records) are bounded.
Resilience & fault tolerance	Graceful degradation, supervised recovery	Database outage $\rightarrow$ JSON-file mode; rainfall outage $\rightarrow$ bundled archive data, then climate baseline with warning; DEM outage $\rightarrow$ synthetic terrain with explicit warning; watchdog restarts the server if <code>/api/health</code> fails. Per-request temp files prevent concurrent requests from overwriting each other's DEM.
Memory & resource management	Bounded resource use	Strided decimation caps Delaunay input below $2N_{max} = 40,000$ points ($159,113 \rightarrow 22,731$ for the reference file); database libraries are imported lazily; the watchdog prunes temporary rasters and rotates logs on a 99%-full disk; numeric thread pools are pinned to the container's single-CPU quota.

Table 3. REST API endpoints.

#	Route	Input	Output	Errors
1	GET /	–	Leaflet single-page app (index.html)	–
2	POST /api/analyze	JSON {lat, lon, village_id?}; $lat \in [-90, 90]$, $lon \in [-180, 180]$	AnalyzeResponse: request id, terrain, rainfall, recommendation, warnings	422 invalid body; 502 DEM unavailable
3	GET /api/analyze/{request_id}	path request_id: int	AnalyzeResponse (from PostGIS, or summary from file history)	404 not found
4	POST /api/analyze-area	JSON {bounds: {min_lat, max_lat, min_lon, max_lon}, name?}	AreaAnalysisResponse: pond site, catchment polygon/area, slope, runoff, rainfall, recommendation, warnings	422 invalid body or min $\geq$ max
5	POST /analyzeContour	multipart field contour_map (.km1/.km2)	ContourAnalysisResponse: as above plus file name, contour count, elevation range	400 wrong type / empty / unparseable; 422 degenerate grid; 500 flow analysis failure
6	GET /api/villages/search	query q (default empty)	{query, results: [{id, name, district, state, lat, lon}]}, max 8	– (network errors degrade silently)
7	GET /api/villages/{id}/history	path village_id: int	{village_id, history: [. . .]} newest first	–
8	GET /api/analyses/recent	–	{analyses: [. . .]}, 12 most recent	–
9	GET /api/health	–	{status, service, database, uptime_seconds, memory_mb, pid}	–

Table 4. Relational schema.

Table	Columns
village	id PK, name, centroid POINT
analysis_request	id PK, village_id FK→village (nullable), point POINT, created_at timestampz
terrain_result	request_id PK/FK, catchment_polygon GEOMETRY, area_km2, avg_slope
rainfall_result	request_id PK/FK, annual_avg_mm, seasonal_json JSON, data_years
pond_recommendation	request_id PK/FK, runoff_m3, depth_m, surface_area_m2, capacity_m3, suitability_score
query_cache	cache_key PK (e.g. "21.25, 81.29"), response_json JSON, created_at

Table 5. Reference survey before and after the audit.

Quantity	Before	After
Pond site	21.24185, 81.28689	21.24171, 81.28692
Catchment area (km ²)	3.8715	3.9116
Mean slope (%)	4.02	3.99
Monsoon rainfall used (mm)	940 (baseline)	1251.4 (Open-Meteo)
Runoff (m ³)	1,273,723	1,713,242
Capacity (m ³)	891,606	1,199,269
Suitability	96.3	98.0

Table 6. Measured latency per stage.

Stage	Time
KML parsing (6.7 MB, 1,355 contours)	0.33 s
Delaunay grid interpolation	0.72 s
Fill, flats, D8, accumulation, site, catchment	0.15 s
Polygonization	0.01 s
Open-Meteo 10-year query	1.25 s
OpenTopography SRTM tile download	11.9 s
/analyzeContour end-to-end (HTTP)	2.6–3.7 s
/api/analyze-area end-to-end	13.1–13.5 s
/api/analyze cold / cached	13.2 s / 8 ms